

What We Built, Step by Step

A beginner-friendly guide to the whole project — what it is, why we did it, what we did in order, and what every confusing word means.

WHAT IS THIS PROJECT, IN ONE SENTENCE?

We took a famous computer science paper about teaching a computer to make predictions from data, rebuilt what it did on a real dataset to check the results were true, and then ran a fair "race" between that method and two newer ones to see which was faster and which was more accurate.

Why This Project Exists

This project was an assignment called **"Tech Fellows: Academic Paper Reproduction + Extension."** The idea behind the assignment is simple: pick a real, published research paper in computer science, and then prove you understand it by doing three things — (1) rebuild the experiment yourself and check you get a similar result, (2) improve or extend it with your own original idea, and (3) explain everything clearly enough that a stranger could follow along and get the same result.

This matters because in the real world, a lot of published research turns out to be hard (or impossible) for other people to repeat — sometimes because of missing details, sometimes because of mistakes nobody caught. Practicing "reproduction" is how working scientists and engineers build trust in results, catch hidden bugs, and build new discoveries on solid ground instead of guesses.

The assignment came with a full plan: set up a clean project (Part 1), reproduce a chosen paper's results (Part 2, the biggest chunk of work), design and build one small original extension (Part 3), and finish with clear documentation, tests, and a short write-up (Part 4) — all saved as 40 separate "save points" on GitHub so the whole journey is visible, not just the final result. That is exactly the structure this document follows below.

WORDS YOU'LL SEE A LOT (READ THIS FIRST)

Machine learning

Teaching a computer to spot patterns in examples, so it can make good guesses about new examples it hasn't seen before.

Dataset

A big table of examples used to teach the model. Like a spreadsheet with rows (people) and columns (facts about them).

Accuracy

Out of 100 guesses, how many the model got right. 87% accuracy means it was correct 87 times out of 100.

Model

The "brain" the computer builds after studying the examples. Once it's trained, you hand it new data and it gives you a prediction.

Training

Showing the model lots of examples so it learns the pattern, like learning multiplication by practicing many problems.

Reproducing a paper

Redoing someone else's experiment yourself, using their instructions, to check you get the same results they claimed.

Extension

Going one step beyond just repeating the original experiment — asking a new question the paper didn't fully answer.

Commit

A saved snapshot of your work, with a note describing what changed — like hitting "save" but keeping every past version.

Data leakage

When info from the test data accidentally "leaks" into training, making the final score look better than it truly is.

GitHub / repository

A website where programmers store their project files online, so anyone can view, download, or copy the work.

Hyperparameters

Settings you choose before training starts, like how deep a decision tree can grow, or how fast the model learns.

Decision tree

A flowchart of yes/no questions used to reach an answer — like playing "20 Questions."

Part 1 Picking the Paper and Setting Up

Before writing any code, we chose what to study and organized a clean workspace for it.

1

Built the project's folders

We created a set of organized folders — one for data, one for code, one for trained models, one for pictures and results, and one for written documents. This is like setting up labeled drawers before a big craft project, so nothing gets lost later.

We also saved this to **GitHub** (a website where programmers store project files online) for the first time, which became the online home base for the whole project.

2

Chose the research paper

We picked a well-known 2016 computer science paper called "**XGBoost: A Scalable Tree Boosting System**." It describes a method computers use to make predictions from tables of data — things like "will this person's income be above or below \$50,000 a year?" based on facts like their age, job, and education.

We chose it because: it doesn't need an expensive computer to run, it's a manageable size to actually understand, and the dataset it uses is free and public.

Part 2 Rebuilding the Original Experiment

This is the "reproduction" — redoing the paper's method ourselves to see if we get similar results.

1

Installed the tools

We made a list of all the software "toolboxes" (called **libraries** — ready-made code someone else wrote that you can reuse) the project would need, and installed them. Like buying all your ingredients before cooking, instead of running to the store mid-recipe.

2

Downloaded and cleaned the data

We wrote code to automatically download a public dataset called **Adult Census Income** — about 45,000 real (but anonymous) people's records: their age, job type, education, and whether they earn more or less than \$50,000 a year.

Raw data is almost never ready to use right away. We had to fix things like missing entries and make sure text categories were labeled in a way the computer could understand.

We also split the data into three piles: a big pile to train on, a small pile to check progress along the way, and a separate pile saved for the final, honest test — never shown to the model while it's learning, so the final score isn't "cheating."

3

Explored the data with charts

Before teaching the computer anything, we made charts and graphs to look at the data ourselves. This step is like reading a recipe fully before cooking, so you're not surprised halfway through.

4

Built the prediction model

We wrote the code for the actual model described in the paper. In simple terms, this method builds many small **decision trees**, and each new tree tries to fix the mistakes the earlier trees made. Combined together, hundreds of these small trees make one strong, accurate prediction.

We also wrote a quick test to make sure the model could actually run before trusting it with real training.

5

Wrote the training and grading code

We built the code that trains the model (shows it the training data over and over until it learns) and the code that grades it afterward — checking how often its predictions were correct on data it had never seen.

We also added a feature to automatically save a backup of the model while it's training — similar to autosave in a word processor.

6

Ran the first training and compared to the paper

We trained the model for real and checked the score against numbers commonly reported for this exact task in other research.

✓ First result: about 87% accuracy — already in the expected range

7

Fine-tuned the settings

Models have adjustable **hyperparameters** — like how deep each decision tree can grow, or how cautious the model should be about trusting patterns too quickly. We tried a couple of combinations, the same way you'd adjust oven temperature and baking time to get a better cake.

8

Found and fixed a real bug

While double-checking our work, we discovered about 52 rows in the dataset were exact duplicates, and a few had accidentally ended up in *both* the training pile and the final test pile — a problem called **data leakage**. It's like a student accidentally seeing a few real exam questions while studying, which makes their practice score look better than their true skill.

We fixed it by removing duplicate rows before splitting the data, so the final test was completely fair.

9

Got the final, trustworthy score

✓ Final result: 87.53% accuracy on data the model had never seen

This matched what other researchers typically report for this exact task, which means the reproduction was a success.

10

Proved it could be repeated by anyone

To make sure this wasn't a fluke, we copied the entire project into a brand-new empty folder (simulating someone downloading it fresh from GitHub) and ran everything again from scratch. It produced the **exact same score**, down to the tiniest decimal, in about 6 seconds — proving the project is genuinely reproducible, not just a lucky one-time result.

11

Wrote it all down

We documented every setting we used, every result we got, and instructions so anyone else could follow the same steps themselves.

Part 3 Going Further: The Extension

Instead of just repeating the paper, we asked a new question it didn't fully answer.

1

Planned a fair comparison

The original 2016 paper didn't just claim its method was accurate — it also claimed the method was fast and efficient. Since then, two newer competing methods have been built specifically to try to beat it: one called **LightGBM** and one called **CatBoost**. We planned an experiment to test all three side-by-side on the exact same data, to see which was really faster, and whether "faster" comes at the cost of accuracy.

2

Built the two new models

We wrote code so the two newer methods could be trained and tested using the exact same process as the original, making the comparison as fair as possible.

3

Ran the race and measured everything

We trained all three methods on identical data and carefully timed: how long each took to learn, how long each took to make predictions afterward, how accurate each was, and how much storage space each saved model took up.

4

Compared the results

METHOD	ACCURACY	TRAINING TIME	PREDICTION SPEED
Original (XGBoost)	87.5%	0.70 sec	middle
LightGBM	87.3%	0.46 sec (fastest)	slowest
CatBoost	87.4%	10.96 sec (slowest)	fastest

The big finding: all three methods were nearly tied on accuracy — less than half a percentage point apart. The real difference was speed. One newer method trained fastest, another was over ten times slower to train but predicted fastest afterward, and the original method landed right in the middle on both.

5

Made charts and wrote up the findings

We created visual charts comparing all three methods and wrote a full explanation of *why* each behaved the way it did, tying results back to how each is built internally.

Part 4 Wrapping Up and Double-Checking

The final stretch: making sure everything works, is explained clearly, and is safe to hand off.

1

Finished all the written explanations

We updated the project's main instructions page, wrote a reflection about what we learned and what surprised us, and made sure every piece of code had a short note explaining what it does and why.

2

Wrote automatic checks ("tests")

We wrote small automatic checks that confirm key parts of the project still work correctly — for example, checking that no duplicate data ever sneaks into both the training and test piles again. Like a smoke detector: it doesn't prevent every problem, but it catches known dangers automatically.

3

Cleaned up the code

We ran a **linter** (a program that automatically checks code for mistakes or messy formatting), tidied everything up, then ran every check one final time to confirm nothing was broken.

4

Published everything to GitHub

We uploaded the entire project — all 40 saved snapshots (**commits**) of progress, from the very first folder to the final polished version — to the public GitHub page, so anyone can view, download, or double-check the work themselves.

5

Wrote the final report

We put together a short written report summarizing the whole project: what the original paper claimed, how we tested it, what we found, and what it means.

THE SHORT VERSION

We picked a well-known machine learning paper, rebuilt its experiment ourselves and got a matching score (87.5% accuracy), found and fixed a real hidden bug along the way, then ran a fair race between that method and two newer competitors to learn that they're all about equally accurate — but very different in speed. Everything is saved, explained, and published online for anyone to check.